Perfect ✅ — here is your **FULL DAY 2 (rebuild) with FREE + PAID toggle + required APIs stack**

👉 This is your **final production-ready version**

🚀 ✅ OVERALL MODEL (YOU WILL FOLLOW FROM NOW)

MODE = "free" or "paid"

👉 Every layer uses router pattern ✅

✅ ✅ REQUIRED CONFIG (START HERE)

📄 **app/config.py**

MODE = "free" # change to "paid" anytime

🚀 ✅ DAY 2 — PARSER LAYER (DUAL MODE)

✅ 🎯 GOAL

PDF → structured elements (titles + text)

✅ 📁 FILES

app/parser/

├── parser_router.py ✅

├── unstructured_parser.py ✅

├── gcp_parser.py ✅ (stub first)

app/ingestion/

├── pipeline.py ✅

✅ ✅ STEP 1 — FREE PARSER

📄 **app/parser/unstructured_parser.py**

```
from unstructured.partition.pdf import partition_pdf


def parse_unstructured(file_path):
  elements = partition_pdf(filename=file_path)

  structured = {
    "titles": [],
    "text": []
  }

  for el in elements:
    text = el.text if hasattr(el, "text") else ""

    if "Title" in str(type(el)):
      structured["titles"].append(text)
    else:
      structured["text"].append(text)

  return structured
```

✅ ✅ STEP 2 — PAID PARSER (GCP)

📄 **app/parser/gcp_parser.py**

👉 (stub now, plug later)

```
def parse_gcp(file_path):
  # TODO: connect GCP Document AI
  return {
    "titles": [],
    "text": []
  }
```

` `

✅ ✅ STEP 3 — ROUTER

📄 **app/parser/parser_router.py**

```
from app.config import MODE
from app.parser.unstructured_parser import parse_unstructured
from app.parser.gcp_parser import parse_gcp


def parse_document(file_path):
  if MODE == "paid":
    return parse_gcp(file_path)


  return parse_unstructured(file_path)
```

✅ ✅ STEP 4 — PIPELINE

📄 **app/ingestion/pipeline.py**

```
from app.parser.parser_router import parse_document


def process_pdf(file_path):
    parsed = parse_document(file_path)


    return {
        "titles": parsed["titles"],
        "text": parsed["text"]
    }
```

``

✅ ✅ TEST

```
python -m tests.test_ingestion
```

✅ ✅ OUTPUT (SAME FOR BOTH MODES)

titles ✅

text ✅

tables ❌ (Day 3)

✅ ✅ NEXT (DAY 3)

text → tables (dual mode)

🚀 ✅ DAY 3 (DUAL PREP – QUICK VIEW)

FREE → your custom table_parser ✅

PAID → GCP DocAI table extract ✅

✅ ✅ REQUIRED API / TECH STACK (BUY THIS)

🏆 ✅ MINIMUM REQUIRED (BEST SETUP)

✅ 1. GCP CORE (REQUIRED)

👉 Enable:

✅ Vertex AI

✅ Document AI

✅ Cloud Storage

✅ 2. MODELS TO USE

✅ LLM

Gemini 1.5 Flash ✅ (cheap default)

Gemini 1.5 Pro ✅ (complex queries)

✅ Embeddings

text-embedding-004

✅ **Parsing**

Document AI → Layout Parser ✅

Document AI → Form Parser ✅

✅ **OCR**

Document AI OCR ✅

✅ ✅ **OTHER (OPTIONAL)**

Qdrant → vector DB ✅

Redis → cache ✅

DuckDB → analytics ✅

✅ ✅ **INSTALL (LOCAL)**

pip install unstructured duckdb fastapi uvicorn

pip install google-cloud-documentai google-cloud-aiplatform

✅ ✅ **AUTH SETUP (GCP)**

gcloud auth application-default login

Set:

GOOGLE_APPLICATION_CREDENTIALS=path_to_json

✅ ✅ **FINAL STACK (YOUR SYSTEM)**

FREE MODE:

Parser: Unstructured

Tables: Custom logic

LLM: Llama 3

PAID MODE:

Parser: Document AI

Tables: Document AI

LLM: Gemini

✅ ✅ HOW YOU WILL USE

✅ TEST FREE

MODE = "free"

✅ TEST PAID

MODE = "paid"

✅ ✅ FINAL FLOW

PDF

↓

Parser (free/paid)

↓

Text ✅

↓

(Day 3 → tables)

✅ ✅ WHERE YOU ARE NOW

Infra ✅

Dual mode ✅

Parsing ✅ (free)

Paid ready ✅ (stub)

Tables ❌ (next)

✅ ✅ NEXT STEP

👉 Say:

Day 3 (dual mode final)

I will give:

- working table logic ✅
- GCP table parser ✅
- clean output ✅

✅ Now you have **production-level architecture already** 🚀